

Objetivo: Praticar os principais comandos do Git e o fluxo de trabalho colaborativo utilizando GitHub, branches, commits e versionamento de código.

Regras da Atividade

- A atividade deverá ser realizada em trio.
- Todos os integrantes devem utilizar Git e GitHub.
- O projeto deverá possuir:
 - repositório remoto no GitHub;
 - branches;
 - commits organizados;
 - arquivo README.md bem estruturado;
 - arquivo .gitignore.
- O link do repositório deverá ser enviado no Classroom da disciplina.

Organização do Trio

Aluno 01 — Líder do Projeto

- Criar o repositório no GitHub;
- Adicionar os outros dois alunos e o professor (pedro.lucas.luz.dev@gmail.com) como colaboradores;
- Criar o arquivo README.md;
- Criar o arquivo .gitignore;
- Adicionar alguns arquivos inúteis para serem ignorados;
- Escrever as questões da atividade;
- Organizar os espaços da pesquisa no README.md.

Aluno 02 — Responsável pelas Respostas

- Criar uma branch própria;
- Responder as questões no README.md;
- Adicionar imagens ilustrativas;
- Criar a pasta img;

- As imagens devem ficar dentro da pasta img;
- A pasta img NÃO deve ser enviada ao GitHub.

Aluno 03 — Responsável pela Pesquisa

- Realizar pesquisas e adicionar nos arquivos padroes-commits.md e padroes-branches.md, utilizando 2 branches diferentes;
- Pesquisar sobre:
 - padrões de commits;
 - padrões de nomenclatura de branches.

Questão 1. Questões Obrigatórias

1. Explique o conceito de controle de versão distribuído e descreva como o Git implementa esse modelo.
2. Diferencie Git e GitHub, destacando suas responsabilidades dentro do processo de desenvolvimento de software.
3. Explique o funcionamento interno de um commit no Git e sua importância para o histórico do projeto.
4. Descreva o fluxo de trabalho do Git envolvendo Working Directory, Staging Area e Repository.
5. Explique o conceito de branch e discuta sua importância em ambientes colaborativos de desenvolvimento.
6. Diferencie os comandos `merge` e `rebase`, apresentando vantagens e desvantagens de cada abordagem.
7. Explique o papel do arquivo `.gitignore` e descreva situações práticas onde sua utilização é indispensável.
8. Descreva o funcionamento dos comandos `git pull` e `git push`, explicando como ocorre a sincronização entre repositórios locais e remotos.
9. Explique o que são conflitos de merge, como eles surgem e quais estratégias podem ser utilizadas para resolvê-los.
10. Discuta a importância do versionamento de código no desenvolvimento moderno de software, considerando manutenção, rastreabilidade e colaboração.
11. Explique a finalidade de Pull Requests no GitHub e sua relevância para revisão de código e integração contínua.
12. Descreva como o Git pode auxiliar na recuperação de versões anteriores de um projeto e na auditoria de alterações realizadas pela equipe.

Questão 2. Comandos Esperados

```
git init
git add .
git commit -m "mensagem"
git push
git pull
git switch -c nome-da-branch
git merge nome-da-branch
```

Questão 3. Exemplo de .gitignore

```
img/
node_modules/
.env
*.log
```

Questão 4. Fluxo Esperado da Atividade

1. Criar o repositório;
2. Clonar o projeto;
3. Criar branches;
4. Realizar alterações;
5. Fazer commits;
6. Enviar para o GitHub;
7. Criar Pull Request;
8. Realizar merge na branch main.

Questão 5. Entrega

- O líder deverá enviar o link do repositório no Classroom;
- Informar o nome completo dos integrantes;
- Todos os integrantes devem possuir commits no projeto.